Primitive-based Truncated Diffusion for Efficient Trajectory Generation of Differential Drive Mobile Manipulators

Long Xu†, Choilam Wong†, Yuhang Zhong, Junxiao Lin, Jialiang Hou, and Fei Gao

***Abstract*— We present a learning-enhanced motion planner for differential drive mobile manipulators to improve efficiency, success rate, and optimality. For task representation encoder, we propose a keypoint sequence extraction module that maps boundary states to 3D space via differentiable forward kinematics. Point clouds and keypoints are encoded separately and fused with attention, enabling effective integration of environment and boundary states information. We also propose a primitive-based truncated diffusion model that samples from a biased distribution. Compared with vanilla diffusion model, this framework improves the efficiency and diversity of the solution. Denoised paths are refined by trajectory optimization to ensure dynamic feasibility and task-specific optimality. In cluttered 3D simulations, our method achieves higher success rate, improved trajectory diversity, and competitive runtime compared to vanilla diffusion and classical baselines. The source code is released at https://github.com/nmoma/nmoma.**

I. Introduction

The development of embodied intelligence [1] has drawn significant attention to differential drive mobile manipulator (DDMoMa), which consists of multi-joint manipulator(s) and a differential drive base. This paper focuses on motion planning, a key component of autonomous navigation in DDMoMa that underpins the efficient and reliable execution of long-horizon tasks.

The objective of motion planning is to generate safe, dynamically feasible, and optimal trajectories guiding DDMoMa from an initial state to a target state. Prevailing methods [2]–[4] rely on numerical optimization. Typical implementations obtain geometrically safe paths utilizing methods combining sampling and searching. The trajectory optimizer then initializes the optimization variables using this path to further derive a dynamically feasible trajectory that is optimal under specified criteria. In simple scenarios, path searching can be ~~very fast~~ because the strategy can be greedy [2]. However, in complex scenarios, dense obstacles shrink the solution space, thus increasing the difficulty of path searching.

Recently, neural networks have been applied to path search problems due to their powerful representation capabilities and GPU-based parallel inference mechanisms, demonstrating remarkable success rates and efficiency on fixed-base manipulator(s) [5] and mobile platforms [6]. However, adapting them to DDMoMa remains challenging to achieve high real-time performance [7], [8]. The primary challenges lie in designing effective and efficient network architectures and training frameworks.

†Indicates equal contribution.
E-mail: {gaolon, fgaoaa}@zju.edu.cn

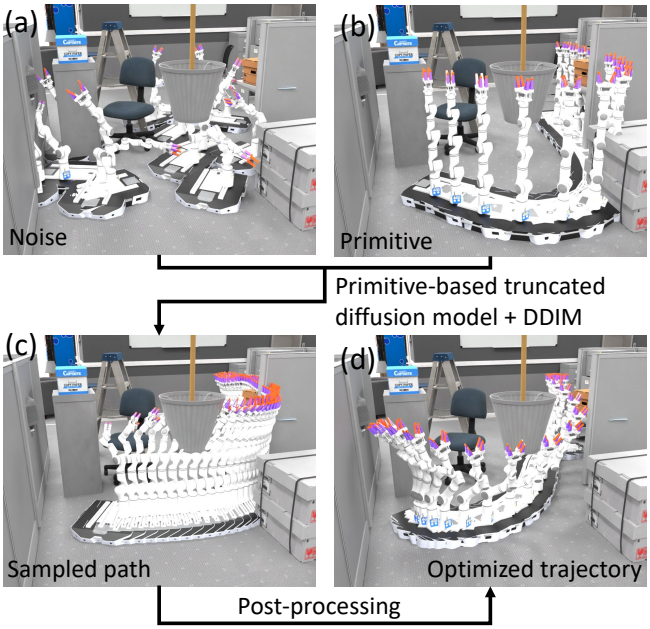

Fig. 1: Process of the proposed planning framework during its deployment. Given the noise (subfigure (a)) and selected primitive (subfigure (b)), paths (subfigure (c)) are sampled via primitive-based truncated diffusion model (PTDM), subsequently post-processed to obtain the final optimized trajectory (subfigure (d)).

Compared with mobile bases or fixed-base manipulators, DDMoMa operates in a much larger 3D workspace and a substantially higher-dimensional configuration space. This requires the neural network to efficiently encode richer information. Moreover, the boundary states (i.e., the start and goal states of DDMoMa) and the environment constitute multimodal input, further complicating network design.

The DDMoMa path search problem is also characterized by numerous local optima. For example, when navigating around wall-like obstacles on the ground, identical manipulator motions may correspond to base paths with different topologies. In contrast, when avoiding suspended obstacles such as chandeliers, base paths with the same topology may still yield distinct manipulator trajectories, such as passing beneath or alongside the obstacle. Diffusion models have recently demonstrated strong capabilities in capturing multi-modal distributions, making them a promising training framework for handling this problem. However, vanilla DDPM [9] has been shown to be susceptible to mode collapse [10], [11], leading to limited path diversity generated by the model. Such diversity is crucial in practice, as

diverse initializations reduce the likelihood of the optimizer being trapped in poor local optima and thereby improve the probability of successful optimization [4].

In this work, we construct a novel motion planning framework to address the aforementioned challenges. For network architecture, to better fuse environment point clouds with the boundary states of the DDMoMa, we propose a keypoint sequence extraction (KSE) module embedded within the task representation encoder. This module maps the boundary states of DDMoMa into 3D space via differentiable forward kinematics. We then separately encode the point cloud and keypoint sequence, employing attention mechanisms for feature fusion. Compared to approaches that either omit KSE or extract features directly in 3D space, our method achieves more effective integration of point cloud and boundary states information.

For training framework, we propose a primitive-based truncated diffusion model (PTDM) that generates DDMoMa paths from a biased distribution instead of sampling directly from a Gaussian distribution as done in the vanilla DDPM [9]. Simulation experiments demonstrate that the proposed strategy enhances both the efficiency and the diversity of generated paths. Fig. 1 shows the data flow of the proposed framework at runtime. The contributions of the paper are:

- A novel task representation encoder for DDMoMa motion planning. It combines attention mechanisms with the proposed differentiable keypoint sequence extraction, enabling effective feature fusion for environment point clouds and the boundary states of DDMoMa.
- A primitive-based truncated diffusion model for DDMoMa motion planning, enhancing the efficiency and path diversity.
- Integrating the above two modules with a optimization-based post-processing, we propose a learning-enhanced planning framework for DDMoMa. Comprehensive evaluations in three types of cluttered 3D simulation environments validate the efficiency of the pipeline.

II. Related Works

A. Classical Motion Planning

Classical motion planning pipelines typically consist of frontend and backend. The frontend is responsible for generating a collision-free path, which serves as an initial guess for the backend. The backend applies trajectory optimization to refine this path, incorporating higher-order dynamics to produce a trajectory that is dynamically feasible.

Sampling-based methods [12] are typically used as the frontend for motion planning of the DDMoMa. They probe the state space with random sampling and incrementally construct a graph representation approximating the free state space, thereby converting path search into a graph search problem. Although they offer desirable theoretical guarantees, such as probabilistic completeness and asymptotic optimality, they are susceptible to the curse of dimensionality [13]. The volume of the configuration space to be approximated grows exponentially with the degrees of freedom (DoF), causing sampling time to become prohibitively long for high-DoF systems, such as DDMoMa, resulting in the difficulty of real-time planning.

To improve the efficiency of the frontend, recent methods [2]–[4], [14] often decompose and plan the paths of mobile base and the mounted robotic arm separately. Yang et al. [3] employ a topo-guided kinodynamic searching method to find the base path. Based on it, they propose a spatiotemporal rapidly exploring random tree to further obtain the whole-body path. Similarly, Wu et al. [2] propose a multilayer constrained RRT*-Connect method to sample the manipulator state for each waypoint from the base path searched by Hybrid-A*. To further improve the efficiency and success rate, Xu et al. [4] adopt the concept of uniform visibility deformation class [15] to generate topological paths, parallel sampling the paths of the manipulator for different base paths. They also propose a novel polynomial-based trajectory representation for efficient trajectory optimization.

B. Learning-enhanced Motion Planning

While the aforementioned greedy decomposition strategy enables real-time motion planning for DDMoMa, it still relies on full state-space sampling as a fallback. In highly complex environments, it inevitably resorts to this time-consuming process [4].

Neural networks' powerful representation capability and natural suitability for parallel computation enable efficient motion planning in complex scenarios. A prominent strategy for accelerating sampling-based planners is to bias the sampler with task and environment-specific prior knowledge [7], [13], [16]. Since the space of feasible paths is highly constrained, concentrating sampling efforts on promising regions yields considerable efficiency gains over a uniform sampling approach. For instance, Ichter et al. [16] utilizes conditional variational autoencoder (CVAE) to learn and sample from the distribution of waypoints conditioned on the planning task and environment.

Similarly, Neural Randomized Planner [7] enhances classical sampling-based methods with a learning-based sampler, aiming to prioritize sampling in promising region given the local environment. Aside from improved efficiency, another key advantage of this enhanced state space sampling paradigm is that, both probabilistic completeness and asymptotic optimality can be retained by combining learned sampler and uniform sampler. However, striking a balance in sampling quality and efficiency is challenging. While an expressive enough model that yields high-quality waypoints could be time consuming, a fast and light-weight model could render subsequent graph search difficult. In addition, adopting the state space paradigm necessitates computationally expensive graph construction and graph search.

In recent years, diffusion models have emerged as a powerful tool for motion planning, capitalizing on their proven ability to capture multi-modal distributions in high-dimensional spaces [8], [10], [17]–[20]. For instance, M^2 Diffuser [8] learns distribution of trajectory conditioned on the environment, and applies guided sampling to sample from

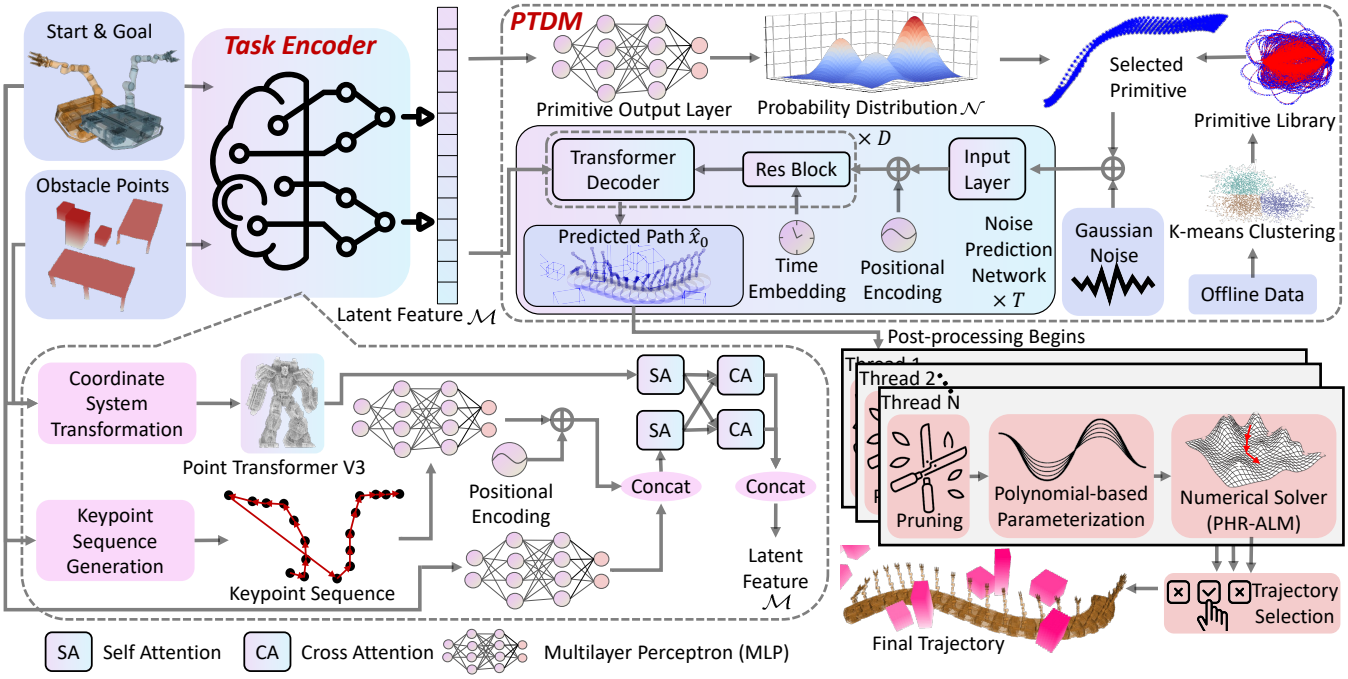

Fig. 2: Proposed planning framework. The neural network encodes the task and samples robot paths efficiently. The paths are then refined by a model-based trajectory optimizer to generate safe, dynamically feasible trajectories.

the posterior distribution. Similarly, MPD [19] learns the marginal distribution and samples from the posterior distribution by enforcing physical and task-related constraints through guided sampling. Such approaches typically require the injection of guidance signals in multiple denoising steps, making it difficult to leverage acceleration techniques such as DDIM [21]. The need for tens or even hundreds of denoising iterations severely limits their real-time applicability. For example, M^2 Diffuser [8] requires over 50 guided denoising steps, resulting in planning times of several seconds per query. To obviate the need for guided sampling and thereby enable the use of acceleration techniques for faster inference, Seo et al. [5] introduce trajectory optimization as post-processing. Their diffusion model learns directly from the posterior distribution, demonstrating efficient motion planning of a fixed-base manipulator in a shelf scenario.

Although acceleration techniques such as DDIM [21] can be employed to speed up the inference of vanilla diffusion models, a reduced number of sampling steps often results in performance degradation. To mitigate this issue, Liao et al. [10] propose an anchor-based truncated diffusion approach, which learns the denoising process from the anchored Gaussian distribution. They demonstrate a balanced trade-off between efficiency and effectiveness in trajectory generation for autonomous driving. Our method is also inspired by the truncated diffusion model [22]. However, unlike [10], our method determines the primitive independently of the diffusion process and prior to its execution. We construct a distribution based on the primitive for the denoising process, rather than denoising the anchored Gaussian distribution composed of all primitives. This approach provides the denoising network with a clearer initial distribution and eliminates the need to sample trajectories for all primitives, thereby reducing computational cost.

III. Planning Framework

Fig. 2 illustrates our planning framework. Upon receiving the obstacle points $\boldsymbol{P}_e \in \mathbb{R}^{N_e \times 3}$ and boundary states $\boldsymbol{s}_s, \boldsymbol{s}_g \in SE(2) \times \mathbb{R}^{N_m}$, the robot analyses the task with the proposed task encoder, where N_e denotes the number of points, N_m denotes the DoF of the manipulator. After selecting motion primitive to sample paths based on latent feature $\mathcal{M}$, it performs model-based post-processing to obtain dynamically feasible trajectories.

A. Task Representation Encoder

The task encoder consists of three components: differentiable preprocessing, feature encoding, and feature fusion, as shown in the bottom left part of the Fig. 2. We perform a coordinate system transformation on the point cloud. Let the start position of the mobile base be $\boldsymbol{p}_s = [x_s, y_s]$ and the goal position be $\boldsymbol{p}_g = [x_g, y_g]$. We define the rotation matrix $\boldsymbol{R}_e \in \mathbb{R}^{3\times3}$ as follows:

$$\theta_d = \text{atan2}(y_g - y_s, x_g - x_s), \tag{1}$$

$$\boldsymbol{R}_e = \begin{bmatrix} \cos\theta_d & -\sin\theta_d & 0 \\ \sin\theta_d & \cos\theta_d & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

Thus, the transformed point cloud is $\boldsymbol{P}_t = (\boldsymbol{P}_s - [\boldsymbol{p}_s, 0])\boldsymbol{R}_e$.

For robot boundary states, using the proposed keypoint sequence extraction (KSE) module, we map them into the 3D space where the point cloud lies, yielding a sequence

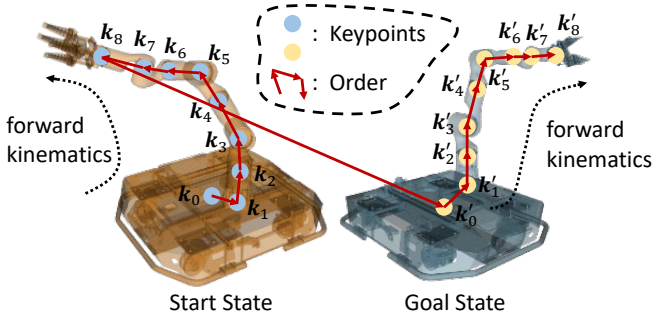

Fig. 3: An example of key point sequence generation for DDMoMa consisting of a 7-DoF manipulator and a mobile base.

of keypoints. As shown in Fig.3, the keypoints correspond to the joint coordinates, the center of the mobile base, and the fixed center of the manipulator. Let the state of the robot $\boldsymbol{s} = [x, y, \theta, \boldsymbol{q}^{\mathrm{T}}]^{\mathrm{T}} \in SE(2) \times \mathbb{R}^{N_m}$, where $\boldsymbol{q} = [q_1, q_2, ..., q_{N_m}]^{\mathrm{T}} \in \mathbb{R}^{N_m}$ represent the joint angles of the manipulator. We obtain keypoints via differentiable forward kinematics with the following equations:

$$\boldsymbol{k}_0 = [x, y, 0, \cos\theta] \triangleq [\boldsymbol{t}_0, c_\theta], \tag{3}$$

$$\boldsymbol{k}_1 = [\boldsymbol{l}_1 \boldsymbol{R}_\theta + \boldsymbol{t}_0, \sin\theta] \triangleq [\boldsymbol{t}_1, s_\theta], \tag{4}$$

$$\boldsymbol{k}_2 = [\boldsymbol{l}_2 + \boldsymbol{t}_1, q_{n1}] \triangleq [\boldsymbol{t}_2, q_{n1}], \tag{5}$$

$$\boldsymbol{k}_3 = [\boldsymbol{l}_3 \boldsymbol{R}_{q_1} + \boldsymbol{t}_2, q_{n2}] \triangleq [\boldsymbol{t}_3, q_{n2}], \tag{6}$$

$$\vdots$$

$$\boldsymbol{k}_j = [\boldsymbol{l}_j \boldsymbol{R}_{q_{j-2}} + \boldsymbol{t}_{j-1}, q_{n(j-1)}] \triangleq [\boldsymbol{t}_j, q_{n(j-1)}], \tag{7}$$

$$\vdots$$

$$\boldsymbol{k}_{N_m+1} = [\boldsymbol{l}_{N_m+1} \boldsymbol{R}_{q_{N_m-1}} + \boldsymbol{t}_{N_m}, q_{nN_m}], \tag{8}$$

where the first three dimensions of each keypoint $\boldsymbol{k}_i, i = 0, 1, ..., N_m + 1$ represent the coordinates, while the last dimension denotes the feature. $\boldsymbol{R}_*, * = \{\theta, q_1, ..., q_{N_m-1}\}$ denote rotation matrices corresponding to rotations about the Z-axis or Y-axis of the keypoint coordinate system. $\boldsymbol{l}_1 \sim \boldsymbol{l}_{N_m+1}$ are link vectors. Referencing the continuous representation [23] for $SE(2)$, we set their features as $\cos\theta$ and $\sin\theta$ respectively. $q_{ni}, i = 1, 2, ..., N_m$ are also normalized to $[-1, 1]$ as follows.

$$q_{ni} = \frac{2q_i - q_{i\max} - q_{i\min}}{q_{i\max} - q_{i\min}}, \tag{9}$$

where $q_{i\min}, q_{i\max}$ are the joint limitations. The order of the keypoint sequence is defined from the base to the end effector, from the start state to the goal state.

Although the keypoints are mapped into the same 3D space as obstacle points, their physical meaning remains fundamentally different. Thus, we employ different encoders to extract their features. Multilayer Perceptron (MLP) and Point Transformer V3 [24] are adopted as keypoint encoder and point encoder, respectively. We utilize self-attention and cross-attention mechanisms for feature fusion, where sinusoidal positional encoding is applied to the keypoint sequences for introducing order information. To enrich the information prior to feature fusion, we also apply MLP directly to robot boundary states, concatenating the results with the keypoint embeddings.

B. PTDM for DDMoMa

Truncated diffusion model has been demonstrated to be a faster and computationally cheaper approach than classical diffusion model [9], [21] in the fields of image generation [22] and autonomous driving [10]. Drawing inspiration from them, in order to improve efficiency and prevent mode collapse [11], we propose a primitive-based truncated diffusion model (PTDM) for motion planning of DDMoMa.

In our work, the path of DDMoMa is expressed as a sequence of states $\boldsymbol{\tau} = [\mathfrak{s}_1, \mathfrak{s}_2, ..., \mathfrak{s}_{N_{\boldsymbol{\tau}}}] \in \mathbb{R}^{(4+N_m)\times N_{\boldsymbol{\tau}}}$, where $\mathfrak{s}_i = [x_i, y_i, c_{\theta i}, s_{\theta i}, \boldsymbol{q}_i^{\mathrm{T}}]^{\mathrm{T}} \in \mathbb{R}^{4+N_m}, i = 1, 2, ..., N_{\boldsymbol{\tau}}$. $N_{\boldsymbol{\tau}}$ is the number of states. As shown in the top right part of the Fig. 2, the PTDM generates the samples by learning the denoising process $p_\psi(\boldsymbol{\tau}^{t-1}|\boldsymbol{\tau}^t, \boldsymbol{c})$ from a noisy-primitive distribution to the original data distribution $p(\boldsymbol{\tau}^0|\boldsymbol{c})$, where the conditions $\boldsymbol{c}$ include $\boldsymbol{P}_e, \boldsymbol{s}_s, \boldsymbol{s}_g$.

The noise distribution predicted by the neural network can be expressed as:

$$p_\psi(\boldsymbol{\tau}^0|\boldsymbol{c}) = \int p(\boldsymbol{\tau}^{\tilde{T}}|\boldsymbol{c}) \prod_{t=1}^{\tilde{T}} p_\psi(\boldsymbol{\tau}^{t-1}|\boldsymbol{\tau}^t, \boldsymbol{c}) d\boldsymbol{\tau}^{1:\tilde{T}}, \tag{10}$$

where $p(\boldsymbol{\tau}^{\tilde{T}})$ is the noisy-primitive distribution. We generate it by truncating the diffusion noise schedule and fusing the truth primitive with Gaussian distribution:

$$p(\boldsymbol{\tau}^{\tilde{T}}) = \sqrt{\bar{\alpha}^{\tilde{T}}} \boldsymbol{\tau}_{\text{prim}} + \sqrt{1 - \bar{\alpha}^{\tilde{T}}} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I}), \tag{11}$$

where $\bar{\alpha}^t = \prod_{i=1}^{t} \alpha^i, \alpha^i \in (0, 1), i \in [1, \tilde{T}]$ are predefined parameters for the noise schedule. $\tilde{T} < T_{\max}$ is the number of truncated diffusion steps, where $T_{\max}$ is the number of original diffusion steps. $\boldsymbol{\tau}_{\text{prim}}$ is the truth primitive, the one in the primitive library with the closest Euclidean distance to the truth path $\boldsymbol{\tau}^0$. Here, we employ the K-Means clustering algorithm for primitive library construction from offline-generated trajectories, which have been demonstrated to be practically effective for primitive generation. [10], [25]

During training, the primitive output layer predicts the probability distribution of the primitives. The target distribution for noise prediction network is

$$q(\boldsymbol{\tau}^t|\boldsymbol{\tau}^{t-1}, \boldsymbol{c}) = \mathcal{N}(\sqrt{\alpha^t}\boldsymbol{\tau}^{t-1}, (1 - \alpha^t)\boldsymbol{I}). \tag{12}$$

C. Post-processing

To generate dynamically feasible trajectories for DDMoMa, we adopt an efficient trajectory representation based on piecewise polynomials and arc length-yaw parameterization [4]. Initializing the trajectories with paths sampled using PTDM, we perform trajectory optimization in parallel, as shown in the bottom right part of Fig. 2.

The trajectory optimization formulation is identical to that in the work [4], except that the optimization variables for

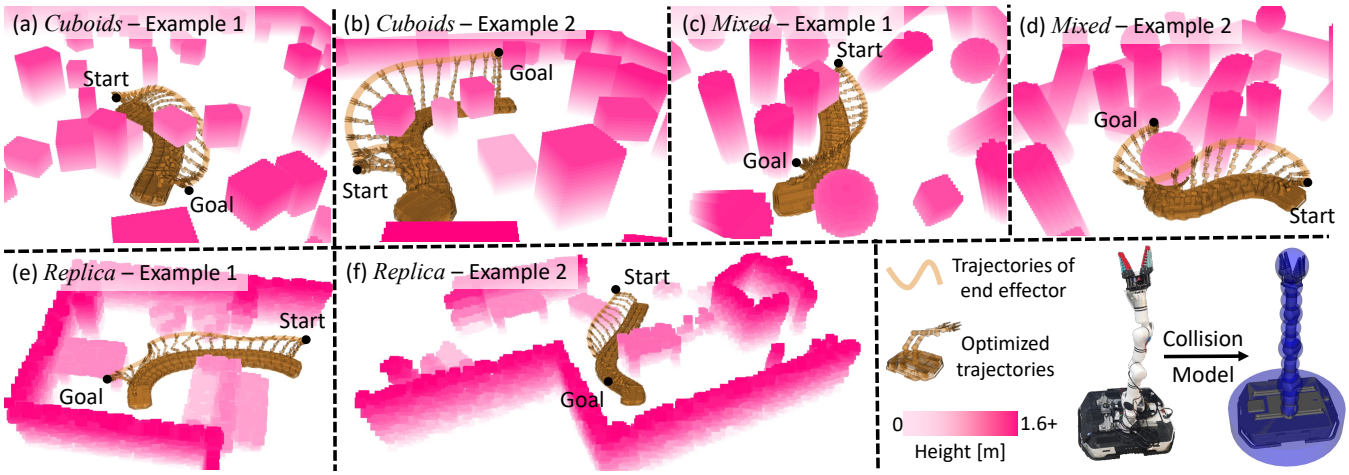

Fig. 4: Robot configuration and simulation environments. Two examples are provided for each environment here, with trajectories generated by the proposed planning framework.

goal state of the mobile base is changed to $[a_f, \theta_f]$, and the constraint is modified to

$$x_g = x_s + \int_0^{t_f} \dot{a}(\tau)\cos\theta(\tau)d\tau, \tag{13}$$

$$y_g = y_s + \int_0^{t_f} \dot{a}(\tau)\sin\theta(\tau)d\tau, \tag{14}$$

where $a(t), \theta(t)$ are the arc length and yaw trajectories, respectively. $t_f > 0$ is the total duration of the trajectory. $a_f = a(t_f), \theta_f = \theta(t_f)$.

To obtain better initial values, inspired by the pruning algorithm in OMPL [12], we perform a pruning step on the sampled paths before optimization. We also adopt the early termination strategy proposed in TopAY [4] to improve efficiency.

IV. Implementation Details

A. Training Losses

To facilitate explicit geometric losses, we choose to directly predict the original path $\boldsymbol{\tau}^0$, instead of the noise $\boldsymbol{\epsilon}$. In this work, the added geometric loss is defined as the expected loss form of the predicted path $\hat{\boldsymbol{\tau}}$: $\bar{\mathcal{L}}_{geo}(\hat{\boldsymbol{\tau}}) = \mathbb{E}_{\boldsymbol{\tau}_{\text{prim}}\sim P_\psi(\boldsymbol{\tau}_{\text{prim}})}[\mathcal{L}_{geo}(\hat{\boldsymbol{\tau}})] = \sum_{\boldsymbol{\tau}_{\text{prim}}} \mathcal{L}_{geo}(\hat{\boldsymbol{\tau}}) P_\psi(\boldsymbol{\tau}_{\text{prim}})$, where $\mathcal{L}_{geo}(\hat{\boldsymbol{\tau}})$ is the weighted sum of following losses.

$$\mathcal{L}_{safe}(\hat{\boldsymbol{\tau}}) = \sum_{i=1}^{N_{\boldsymbol{\tau}}} \sum_{j=1}^{N_c} \frac{\text{ReLU}(r_j^{thr} - \text{SDF}(\textbf{ColliPts}(\hat{\boldsymbol{s}}_i, j)))}{N_{\boldsymbol{\tau}}},$$

$$\mathcal{L}_{smooth}(\hat{\boldsymbol{\tau}}) = \sum_{j\in\mathcal{S}} \text{ReLU}\Big(\sum_{i=1}^{N_{\boldsymbol{\tau}}-1} \Delta_{j,i} - \sum_{i=1}^{N_{\boldsymbol{\tau}}-1} \Delta^*_{j,i}\Big), \tag{15}$$

$$\mathcal{L}_{unip}(\hat{\boldsymbol{\tau}}) = \frac{\text{Std}([\Delta_{p,1}, \Delta_{p,2}, ..., \Delta_{p,N_{\boldsymbol{\tau}}}])}{\sqrt{N_{\boldsymbol{\tau}-1} \sum_{i=1}^{N_{\boldsymbol{\tau}}-1} \Delta^*_{p,i}}}, \tag{16}$$

where $\mathcal{L}_{safe}$ is obstacle avoidance loss. The function $\textbf{ColliPts}(\boldsymbol{x}, y)$ maps the state $\boldsymbol{x} \in \mathbb{R}^{4+N_m}$ and index $y \in \mathbb{Z}_+$ to collision detection point $\boldsymbol{c}_{\boldsymbol{x},y} \in \mathbb{R}^3$. As shown by the robot collision model in Fig. 4, we approximate the collision geometry using a set of cylinders and spheres. r_j^{thr} is the safe threshold corresponding to the j-th point. The function SDF : $\mathbb{R}^3 \mapsto \mathbb{R}$ is Signed Distance Field (SDF) corresponding to the environment. In SDF, the value at each state of the space is the distance to the edge of the nearest obstacle, which is negative inside the obstacles.

Eq.(15) denotes smoothness loss, where $\mathcal{S} = \{p, \theta, \boldsymbol{q}\}$. We represent smoothness by combining the arc length of the base path, the total change in the yaw angle of the base, and the joint angles of the manipulator.

$$\Delta_{p,i} = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}, \tag{17}$$

$$\Delta_{\theta,i} = \sqrt{(c_{\theta(i+1)} - c_{\theta i})^2 + (s_{\theta(i+1)} - s_{\theta i})^2}, \tag{18}$$

$$\Delta_{\boldsymbol{q},i} = \sum_{j=1}^{N_m} |q_{j,i+1} - q_{j,i}|. \tag{19}$$

$\Delta_{j,i}, \Delta^*_{j,i}$ denote the values that correspond to the predicted path $\hat{\boldsymbol{\tau}}$ and the gourd truth $\boldsymbol{\tau}^0$.

Eq.(16) denotes the uniform loss. In this work, we perform equal-interval arc length sampling on the trajectory optimized by classical trajectory planner to generate the ground truth. Thus, we penalize the normalized standard deviation to meet this condition. In addition, focal loss [26] is chosen for primitive classification.

B. Data Collection

Following the work [4], we generate hundreds of random environments and random start-goal states to construct motion planning tasks. For data collection, we employ an efficient planner TopAY [4] and adjust its parameters to allow sufficient time for path searching and trajectory optimization. To compensate for incompleteness of the front end in TopAY [4], we adopt OMPL [12] to sample the path in full state space of DDMoMA when TopAY [4] fails. Optimized trajectories are checked to ensure dynamic feasibility and safety, then recorded with corresponding boundary states, obstacle point clouds, and SDF.

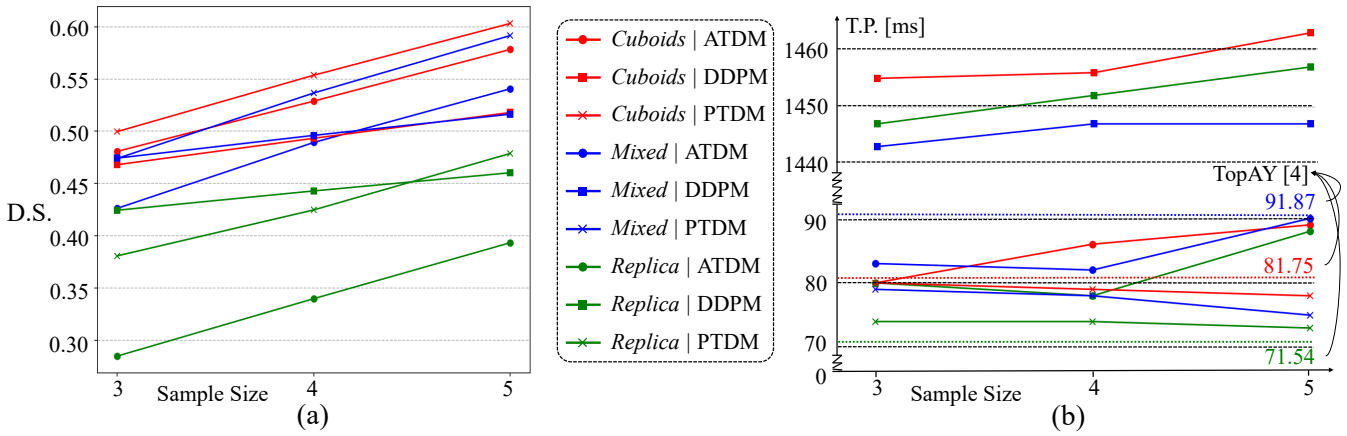

Fig. 5: Comparisons of diversity score (D.S., subfigure (a)) and planning time (T.P., subfigure (b)) across different diffusion strategies as the number of sampled paths varies in different environments. In most of cases, PTDM achieves lower planning time consuming and higher diversity than vanilla DDPM and anchor-based diffusion (ATDM). The red, blue, and green dashed lines in subfigure (b) represent the average T.P. of TopAY [4] in *Cuboids*, *Mixed*, and *Replica*, respectively.

V. Results

A. Experimental Setup

The DDMoMa used in TopAY [4] is employed in this work to evaluate the proposed method. It consists of a two-wheeled differential drive mobile base and a 7-DoF manipulator, as shown in the bottom right corner of Fig. 4. We employ three different kinds of scenes in simulation for benchmark comparisons, which are denoted by *Cuboids*, *Mixed*, and *Replica*, as shown in Fig. 4(a)-(f). Each are $10m \times 10m$ walled rooms filled with randomly placed obstacles of random size. For *Cuboids*, there are 20 grounded cuboids and 30 floating cuboids. To increase the environmental complexity, we introduce additional types of geometric obstacles, forming the *Mixed*, which includes 10 cuboids, 10 spheres, and 25 cylinders. To perform testing in more realistic scenarios, we adopt the ReplicaCAD dataset [27], randomly cutting it to obtain different environments.

For each type of scenario, we randomly sample one million environment–trajectory pairs as the training set and evaluate the model in one thousand unseen environments. All training and evaluations are run on Ubuntu 20.04 with an Intel i9-14900 CPU and a GeForce RTX 4090D GPU. We use the Adam [28] optimizer with a learning rate 1.0e-4 to update the model parameters. The weights of $\mathcal{L}_{safe}$, $\mathcal{L}_{smooth}$, $\mathcal{L}_{unip}$ are set to 50.0, 0.1, and 20.0, respectively. We stack 2 cascade diffusion transformer layers [29] for PTDM, where the number of the primitives is set to 32, the sequence length of the robot path $N_{\boldsymbol{\tau}} = 64$. Following the configuration for anchor-based diffusion [10], the training diffusion schedule is truncated by $50/1200$ to diffuse the primitive. During inference, only 2 denoising steps are used for evaluation.

B. Evaluation Metrics

We use following four kinds of metrics to evaluate the performance of the planner.

- *Diversity score of the paths* (D.S.): Following the work [10], we use the method based on mean Intersection over Union (mIoU) to evaluate the diversity of the paths generated by the model. Specifically, the diversity score of a single sampling result is

$$D.S. = 1 - \frac{1}{N}\sum_{i=1}^{N}\frac{\text{Vol}(\mathfrak{T}_i \cap \bigcup_{j=1}^{N}\mathfrak{T}_j)}{\text{Vol}(\mathfrak{T}_i \cup \bigcup_{j=1}^{N}\mathfrak{T}_j)}, \tag{20}$$

  where $\mathfrak{T}_i, i = 1, 2, ..., N$ represents the swept body of i-th denoised trajectory, $\bigcup_{j=1}^{N}\mathfrak{T}_j$ is the union of all of them. The function $\text{Vol}(\cdot)$ is used to calculate volume. A higher score indicates higher diversity. In this work, to evaluate more efficiently, we approximate the swept body by the collision model of the DDMoMa.
- *Success rate of the planning* (S.R./%): Planning attempts will be considered failed if either the planner cannot find feasible path within limited time or none of the optimization processes successfully return with dynamically feasible and collision-free trajectories.
- *Time consumption of planning* (T.P./ms): It consists of the time consumed by the path finding in frontend and optimization-based backend.
- *Trajectory duration* (T.D./s): The total time required for a robot to execute a planned motion from start to goal, which reflects the optimality of the trajectory.

C. Quantitative Evaluation

In the following experiments, we seek to evaluate the following claims:

- *Claim 1.* Proposed primitive-based truncated diffusion can enhance the efficiency and diversity of the paths.
- *Claim 2.* Proposed task representation encoder that combines the attention mechanisms with KSE can achieve more effective feature fusion for the environment point clouds and the boundary states of DDMoMa, thereby improving the success rate and optimality of planning.

- *Claim 3.* Proposed learning-enhanced planner can achieve higher success rate and efficiency than state-of-the-art (SoTA) classical method.

To validate *Claim 1*, we conduct comparisons of diversity and time consumption between PTDM, vanilla [9] and anchor-based [10] diffusion models. Fig. 5 shows the line charts of average T.P and D.S. regarding the sample size of the diffusion model. As shown in Fig. 5(a), PTDM significantly enhances the diversity of the sampled results. We attribute this improvement to the clearer distribution provided to the denoising module, which more effectively prevents mode collapse [10]. As shown in Fig. 5(b), by leveraging DDIM, which greatly reduces the number of inference steps, PTDM achieves an order-of-magnitude improvement in efficiency compared with vanilla diffusion model. The T.P. of anchor-based diffusion is longer than that of PTDM and increases sharply as the sample size grows from 4 to 5. This is mainly because it requires sampling for all primitives, resulting in substantial computational cost. Interestingly, the T.P. of PTDM slightly decreases as the number of samples increases. This phenomenon also occurs in anchor-based diffusion in *Mixed* and *Replica*, when the sample size increases from 3 to 4. We speculate that the greater variety of trajectories provides a higher probability of good initialization, thereby reducing optimization time consumption and enabling earlier termination of post-processing.

To validate *Claim 2*, we conduct comparisons between proposed task encoder and the following task encoders:

- *Pspace*: KSE is employed, but instead of encoding the keypoint sequence separately, it is integrated into the obstacle point cloud. A new four-dimensional feature $[o, e_1, e_2, e_3]$ is added to distinguish keypoints from obstacles. Here, $o = 0, 1, ..., N_m + 1$ represents the keypoint order, while obstacle points are assigned a value of -100 in this dimension. Using one-hot encoding, $[e_1, e_2, e_3] = [1, 0, 0] \vee [0, 1, 0] \vee [0, 0, 1]$ denote the obstacles, start state, and goal state, respectively.
- *Attention*: KSE is not used. Instead, MLP is adopted directly for feature extraction of boundary states. These features are then fused with the features from obstacle point cloud through the cross attention mechanism.

The results are shown in Fig. 6, where the path ratio (P.R.) denotes the ratio between the T.D. of the learning-enhanced planner and that of SoTA classical method, TopAY [4], in the same task. A lower P.R. indicates better optimality, and a P.R. less than 1 signifies that the result outperforms TopAY [4]. As shown in Fig. 6, the proposed task encoder achieves the highest success rate in most cases in *Cuboids* and *Mixed*, and also outperforms other encoders in terms of P.R.

Although *Pspace* incorporates boundary states into the task encoder and maps them into the obstacle point cloud space, their substantial differences in physical meaning may cause the encoder designed specifically for point clouds to lose useful information during downsampling, making it difficult to extract effective features directly from the fused point cloud. In contrast, the proposed method encodes each keypoint using MLP without downsampling and fuses features with the obstacles through a cross attention mechanism, achieving superior performance. While *Attention* and *Proposed* achieve comparable S.R. across different environments, *Proposed* consistently achieves a considerable advantage over *Attention* in terms of optimality, as indicated by lower P.R., further validating the effectiveness of KSE.

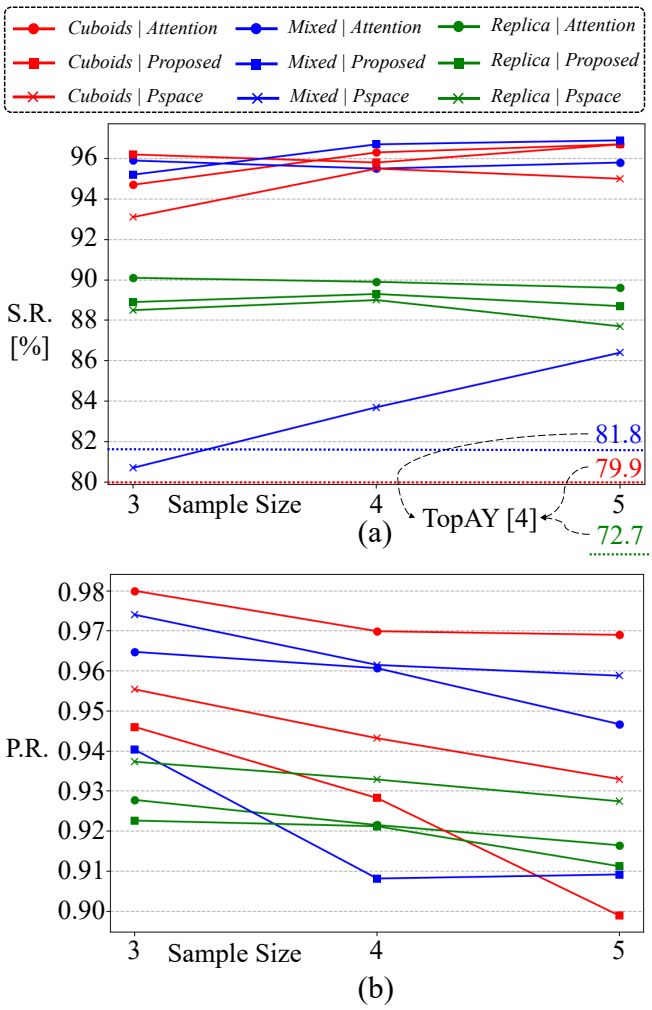

Fig. 6: Ablation study on task encoders: success rate (S.R.) and path ratio (P.R.) in three types of environments. The red, blue, and green dashed lines in subfigure (a) represent the average S.R. of TopAY [4] in *Cuboids*, *Mixed*, and *Replica*, respectively.

To validate *Claim 3*, we compare the proposed planner with SoTA classical method, TopAY [4]. As shown in Fig. 5(b) and Fig. 6, our method outperforms TopAY [4] in terms of success rate, efficiency, and optimality, demonstrating the effectiveness and efficiency of the proposed framework in addressing the motion planning problem of DDMoMa. More demonstrations and more detailed comparisons between different task encoders and diffusion models can be found on our project page[1].

[1]https://nmoma.github.io/nmoma/

VI. Conclusion & Limitations

In this paper, we present a learning-enhanced motion planning framework for DDMoMa. Benefiting from proposed KSE and PTDM, in cluttered 3D simulation environments, our method achieves higher success rates and trajectory diversity compared to vanilla diffusion models and classical baselines, while maintaining competitive runtime efficiency.

Despite these promising results, several limitations remain to be addressed for a broader application. Our validation is currently restricted to static simulation environments, and the gap between simulation and the physical world, which is characterized by perception noise and control uncertainty, has not yet been tested. Limited onboard computing resources may also lead to reduced efficiency. Furthermore, our current diffusion process is conducted at discrete path points, still requiring trajectory optimization-based post-processing to obtain dynamically feasible trajectories, thereby increasing the complexity of the framework.

In the future, we will try to incorporate advanced diffusion frameworks, such as EDM [30], Consistency model [31], and perform diffusion directly in the trajectory representation space to further accelerate inference and improve success rate [32]. We also plan to deploy the planner on physical robots to test and bridge the potential sim-to-real gap. To further exploit the advantages of the efficiency of our method, we will also consider adapting it to mobile manipulators with different configurations and more environments with different levels of difficulty.

References

[1] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, "Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=tQJYKwc3n4

[2] C. Wu, R. Wang, M. Song, F. Gao, J. Mei, and B. Zhou, "Real-time whole-body motion planning for mobile manipulators using environment-adaptive search and spatial-temporal optimization," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 1369–1375.

[3] Y. Yang, F. Meng, Z. Meng, and C. Yang, "Rampage: Toward whole-body, real-time, and agile motion planning in unknown cluttered environments for mobile manipulators," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 11, pp. 14 492–14 502, 2024.

[4] L. Xu, C. Wong, M. Zhang, J. Lin, J. Hou, and F. Gao, "Topay: Efficient trajectory planning for differential drive mobile manipulators via topological paths search and arc length-yaw parameterization," *arXiv preprint arXiv:2507.02761*, 2025.

[5] M. Seo, Y. Cho, Y. Sung, P. Stone, Y. Zhu, and B. Kim, "Presto: Fast motion planning using diffusion models based on key-configuration environment representation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 861–10 867.

[6] Z. Han, M. Tian, Z. Gongye, D. Xue, J. Xing, Q. Wang, Y. Gao, J. Wang, C. Xu, and F. Gao, "Hierarchically depicting vehicle trajectory with stability in complex environments," *Science Robotics*, vol. 10, no. 103, p. eads4551, 2025.

[7] Y. Lu, Y. Ma, D. Hsu, and P. Cai, "Neural randomized planning for whole body robot motion," *arXiv preprint arXiv:2405.11317*, 2024.

[8] S. Yan, Z. Zhang, M. Han, Z. Wang, Q. Xie, Z. Li, Z. Li, H. Liu, X. Wang, and S.-C. Zhu, "M 2 diffuser: Diffusion-based trajectory optimization for mobile manipulation in 3d scenes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[9] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[10] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang *et al.*, "Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 12 037–12 047.

[11] R. Barceló, C. Alcázar, and F. Tobar, "Avoiding mode collapse in diffusion models fine-tuned with reinforcement learning," *arXiv preprint arXiv:2410.08315*, 2024.

[12] I. A. Sucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[13] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 48–66, 2020.

[14] S. Thakar, P. Rajendran, A. M. Kabir, and S. K. Gupta, "Manipulator motion planning for part pickup and transport operations from a moving base," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 1, pp. 191–206, 2020.

[15] B. Zhou, F. Gao, J. Pan, and S. Shen, "Robust real-time uav replanning using guided gradient-based optimization and topological paths," in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 1208–1214.

[16] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7087–7094.

[17] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[18] M. Sharma, A. Fishman, V. Kumar, C. Paxton, and O. Kroemer, "Cascaded diffusion models for neural motion planning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 14 361–14 368.

[19] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters, "Motion planning diffusion: Learning and adapting robot motion planning with diffusion models," *IEEE Transactions on Robotics*, vol. 41, pp. 4881–4901, 2025.

[20] H. Ren, Y. Zeng, Z. Bi, Z. Wan, J. Huang, and H. Cheng, "Prior does matter: Visual navigation via denoising diffusion bridge models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 12 100–12 110.

[21] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=St1giarCHLP

[22] H. Zheng, P. He, W. Chen, and M. Zhou, "Truncated diffusion probabilistic models and diffusion-based adversarial auto-encoders," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=HDxgaKk956l

[23] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 5745–5753.

[24] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer v3: Simpler faster stronger," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 4840–4851.

[25] Z. Han, L. Xu, L. Pei, and F. Gao, "Dynamically feasible trajectory generation with optimization-embedded networks for autonomous flight," *IEEE Robotics and Automation Letters*, vol. 10, no. 10, pp. 9995–10 002, 2025.

[26] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2980–2988.

[27] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. Chaplot, O. Maksymets, A. Gokaslan, V. Vondrus, S. Dharur, F. Meier, W. Galuba, A. Chang, Z. Kira, V. Koltun, J. Malik, M. Savva, and D. Batra, "Habitat 2.0: Training home assistants to rearrange their habitat," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[29] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF international conference on computer*

vision, 2023, pp. 4195–4205.

[30] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the design space of diffusion-based generative models," *Advances in neural information processing systems*, vol. 35, pp. 26 565–26 577, 2022.

[31] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," in *ICML*, 2023, pp. 32 211–32 252. [Online]. Available: https://proceedings.mlr.press/v202/song23a.html

[32] A. Srikanth, P. Mahajan, K. Saha, V. Mandadi, P. Paul, P. Wadhwani, B. Bhowmick, A. Singh, and M. Krishna, "Gpd: Guided polynomial diffusion for motion planning," in *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*. IEEE, 2025, pp. 2758–2765.